Applied Information Extraction in Python

Today's goal • 3/3 items

Module 1
Information Extraction

Module 2
Named Entity Recognition (NER)

Module 3
Neural Network Models

What Is Deep Learning?

- Introduction to Deep Learning
  Video • 4 min
- Understanding Deep Learning: A Shift From Rules to Representation
  Reading • 10 min
- Knowledge Check: What Is Deep Learning?
  Practice Assignment • Grade: 100%

What Are Neural Network Models?

- Neural Network Models
  Video • 7 min
- **Activation Functions: How Deep Learning Models Make Decisions**
  Reading • 10 min
- Knowledge Check: What Are Neural Network Models?
  Practice Assignment • 10 min

Deeper Neural Networks

- Deep Neural Network Models
  Video • 6 min
- Understanding Deep Neural Network Models: How Depth Enables Learning at Scale
  Reading • 10 min
- Knowledge Check: Deeper Neural Networks
  Practice Assignment • 10 min

How to Build IE Models Using DNNs

- Building an Information Extraction Pipeline with BiLSTMs and CRFs
  Reading • 10 min
- Jupyter Notebook Practice on Training LSTMs
  Lab • 1h
- Demo: Configuring the Bi-Directional LSTM
  Video • 4 min

Module Wrap-Up & Assignment

- Assignment 3 Introduction: Building an Information Extraction Pipeline with BiLSTMs and CRFs
  Video • 3 min
- Build an Information Extraction Pipeline using Deep Neural Networks
  Programming Assignment • 3h
- Module 3 Assignment
  Graded Assignment • 30 min

Module 4
Transformers Transform Information Extraction

Activation Functions: How Deep Learning Models Make Decisions

0:26 / 8:19 2x

What Is an Activation Function?

In the last lesson, you were introduced to the basic building block of deep learning: the **neuron**. A neuron takes a set of inputs, multiplies them by learned weights, sums the result, and passes it through a function to decide whether to "activate" or not.

That last step, the **activation function**, is what gives deep learning its name.

Without it, neural networks would just be stacks of linear equations. With it, they can model complex, non-linear relationships that traditional models can't easily capture.

Mathematically Speaking

Each neuron computes:

output = $f(w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n)$

Where:

- The weighted sum is the neuron's **raw signal**
- The function **f( )** is the **activation function**
- It decides whether to "fire" and by how much

Without a non-linear activation, all layers in your model collapse into one, and you're stuck modeling linear relationships, no matter how deep the network.

Why Non-Linearity Matters

Real-world problems are rarely linear. For example:

- The relationship between **age**, **BMI**, and **risk of diabetes**
- The detection of **sarcasm** in a sentence
- The likelihood that **"University of Michigan"** is an organization

These can't be captured by simple linear equations. **Activation functions introduce non-linearity**, allowing neural networks to represent:

- Curves
- Thresholds
- Discontinuous or nonlinear patterns

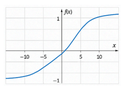
Common Activation Functions (And When to Use Them)

Let's walk through the most widely used activation functions:

1. Sigmoid

Figure 1 shows a Sigmoid function.

Figure 1. A Sigmoid function plot.



$f(x) = \frac{1}{1+e^{-x}}$

Output range: (0, 1)

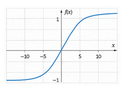
- Smooth, S-shaped curve
- Historically popular in early neural networks
- Problem: outputs saturate near 0 or 1 → causes vanishing gradients

When to use: Often used in the output layer for binary classification.

2. Tanh (Hyperbolic Tangent)

Figure 2 shows a Tanh (Hyperbolic Tangent) function.

Figure 2. A Tanh (Hyperbolic Tangent) plot.



$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

Output range: (−1, 1)

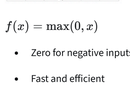
- Like sigmoid, but centered at 0
- Slightly better gradient flow
- Still suffers from vanishing gradients

When to use: In legacy models or when negative outputs are needed

3. ReLU (Rectified Linear Unit)

Figure 3 shows a ReLU (Rectified Linear Unit) function.

Figure 3. A ReLU (Rectified Linear Unit) plot.



$f(x) = \max(0, x)$

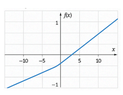
- Zero for negative inputs; linear for positive
- Fast and efficient
- Doesn't saturate on the positive side → avoids vanishing gradients

When to use: Default choice for **hidden layers** in deep networks.

4. Leaky ReLU

Figure 4 shows a Leaky ReLU (Rectified Linear Unit) function.

Figure 4. A Leaky ReLU plot.



$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{otherwise} (\text{where } \alpha \text{ is a small constant, e.g., } 0.01) \end{cases}$$

- Allows small negative outputs
- Fixes the "**dead neuron**" problem of ReLU

When to use: Improves stability in deeper networks.

5. ELU (Exponential Linear Unit)

- Like Leaky ReLU, but smoother for negative values
- Promotes faster convergence and better generalization

When to use: In advanced models requiring performance fine-tuning

How They Fit into the Bigger Picture

Each neuron in a network applies an activation function after combining its inputs.

In deep networks, stacking activations across layers allows the model to learn:

- **Hierarchical patterns** (e.g., letters → words → phrases in NLP)
- **Spatial features** (e.g., edges → shapes → objects in images)

The choice of activation affects:

- **Model speed**
- **Training efficiency**
- **Gradient flow** during back propagation
- **Final accuracy and generalization**

Want to See Them in Action?

- **CS231n Lecture Notes – Activation Functions Overview** (Stanford)
- **Interactive Plot of Sigmoid, Tanh, and ReLU** (TensorFlow Playground)
- **ReLU vs Sigmoid Explained Visually – 3Blue1Brown (YouTube)**

Reflection Prompt

Try sketching or plotting a few activation functions on paper:

- How does each one handle **negative inputs**?
- What happens if most outputs become **zero or very small** across a layer?

Think about how this affects learning as you build deeper models.

Assignment Preview

In your next lessons, you'll **stack neurons into full networks**, where choosing the right activation function can make or break your model's performance.

Think of activation functions as the **decision-makers** in your network, simple but essential.

Go to next item ✓ Completed

Like Dislike Report an issue

Go to next item →